

Санкт-Петербургский национальный исследовательский университет ИТМО  
Мегафакультет трансляционных информационных технологий  
Факультет информационных технологий и программирования

## Лабораторная работа №4

по дисциплине «Проектирование баз данных»

Выполнил студент группы М3212  
Молчанов Ростислав

Проверил  
Папикян Сергей Седракович

Санкт-Петербург  
2026

## Цель работы

Изучить средства маскировки и анонимизации данных в PostgreSQL с использованием расширения PostgreSQL Anonymizer. Настроить динамическое маскирование чувствительных полей в таблицах и представлениях, а также подготовить анонимизированные материализованные представления для аналитических запросов.

## 1. Подготовка окружения

Для выполнения работы использован Docker-образ PostgreSQL Anonymizer. Запуск базы данных настроен в compose-файле с предзагрузкой библиотеки `anon`:

```
services:
  postgres:
    image: registry.gitlab.com/dalibo/postgresql_anonymizer:stable
    command:
      - postgres
      - -c
      - shared_preload_libraries=anon
```

Дополнительно при инициализации базы данных выполняется подключение и настройка расширения:

```
CREATE EXTENSION IF NOT EXISTS anon CASCADE;
SELECT anon.init();
```

Базовая схема данных и тестовое наполнение были взяты из предыдущих лабораторных работ. В схеме присутствуют таблицы `users`, `employees`, `orders`, `products`, `order_items`, `facilities`, `pickup_points` и другие. Данные загружаются из CSV-файлов при помощи команды `\copy`.

## 2. Динамическое маскирование

### 2.1. Выбор чувствительных атрибутов

Для динамического маскирования были выбраны поля, содержащие персональные данные:

- `users.full_name`;
- `users.email`;
- `users.phone`;
- `employees.full_name`;
- `employees.email`;
- `employees.phone`.

## 2.2. Настройка правил

Правила маскирования заданы через `SECURITY LABEL FOR anon`. Для разных типов данных использованы разные способы сокрытия:

- полная замена значения константой;
- частичное скрывание строки;
- маскирование функцией.

Пример настроенных правил:

```
SECURITY LABEL FOR anon ON COLUMN users.full_name  
IS 'MASKED WITH VALUE $$CONFIDENTIAL$$';
```

```
SECURITY LABEL FOR anon ON COLUMN users.email  
IS 'MASKED WITH FUNCTION anon.partial_email(email)';
```

```
SECURITY LABEL FOR anon ON COLUMN users.phone  
IS 'MASKED WITH FUNCTION anon.partial(phone,2,$$*****$$,2)';
```

Аналогичные правила заданы и для таблицы `employees`. Отдельно были добавлены правила для поля `user_name` во `v_order_details` и `mv_order_details`, поскольку маскировка базовых таблиц не распространяется на представления автоматически.

## 2.3. Настройка роли с маскированием

Для проверки работы механизма создана отдельная роль `analyst_masked`. Для этой роли включено прозрачное динамическое маскирование:

```
ALTER ROLE analyst_masked SET anon.transparent_dynamic_masking = true;  
SECURITY LABEL FOR anon ON ROLE analyst_masked IS 'MASKED';
```

После этого один и тот же запрос возвращает разные результаты в зависимости от роли: обычный пользователь видит исходные значения, а `analyst_masked` — уже замаскированные.

## 3. Анонимизация материализованных представлений

В работе были переопределены два материализованных представления из предыдущей лабораторной работы:

- `mv_order_details`;
- `mv_product_sales`.

Их задача — сохранить аналитическую ценность данных, но убрать возможность прямой идентификации пользователей, заказов и точных коммерческих показателей.

### 3.1. mv\_order\_details

Представление строится на основе таблиц `orders`, `users` и `pickup_points`. В нём используются следующие преобразования:

- **Generic Hashing:** идентификаторы заказа и пользователя заменены сокращённым хешем;
- **Generalization:** дата заказа обобщена до месяца;
- **Generalization:** сумма заказа представлена интервалом;
- **Pseudonymization:** имя пользователя заменено устойчивым псевдонимом;
- **Generalization:** идентификатор пункта выдачи переведён в диапазон.

Пример:

```
CREATE MATERIALIZED VIEW mv_order_details AS
SELECT
    left(anon.hash(o.id::text), 16) AS order_ref,
    anon.generalize_tsrage(
        o.created_at AT TIME ZONE 'Europe/Moscow',
        'month'
    ) AS created_at_month,
    anon.generalize_numrange(o.total, 500) AS total_bucket,
    left(anon.hash(u.id::text), 16) AS user_ref,
    concat(
        anon.pseudo_first_name(u.id, 'lab4-order-details'),
        ' ',
        anon.pseudo_last_name(u.id, 'lab4-order-details')
    ) AS user_name
FROM orders o
JOIN users u ON u.id = o.user_id;
```

В результате аналитик может изучать динамику заказов, распределение по месяцам и диапазонам сумм, но не получает доступ к исходным идентификаторам и персональным данным.

### 3.2. mv\_product\_sales

Представление агрегирует продажи по товарам и категориям. Для него были выбраны следующие методы анонимизации:

- **Generic Hashing:** скрытие идентификатора товара;
- **Generalization:** цена товара представлена диапазоном;

- **Generalization:** количество проданных единиц представлено диапазоном;
- **Adding Noise:** к суммарной выручке добавлен шум;
- **Generalization:** количество заказов и дата последней продажи также обобщены.

Пример:

```
CREATE MATERIALIZED VIEW mv_product_sales AS
SELECT
    left(anon.hash(p.id::text), 16) AS product_ref,
    c.name AS category_name,
    anon.generalize_numrange(p.price, 500) AS price_bucket,
    anon.generalize_int8range(SUM(oi.quantity)::bigint, 25)
        AS total_units_sold_bucket,
    ROUND(
        anon.noise(SUM((oi.quantity * oi.price_at_order)::numeric), 0.05),
        2
    ) AS approx_total_revenue
FROM order_items oi
JOIN products p ON p.id = oi.product_id
JOIN categories c ON c.id = p.category_id
GROUP BY p.id, c.name, p.price;
```

Это представление позволяет анализировать продажи по категориям и ценовым сегментам, не раскрывая точные финансовые показатели.

## 4. Использованные стратегии анонимизации

В ходе работы были применены следующие стратегии:

- **Generalization** — замена точных значений диапазонами или укрупнёнными временными интервалами;
- **Generic Hashing** — замена идентификаторов хешами;
- **Pseudonymization** — подмена реальных имён устойчивыми псевдонимами;
- **Adding Noise** — внесение небольшого шума в числовые агрегаты.

Кроме того, для динамического маскирования применялись полная замена константой и частичное скрывание строк.

## 5. Проверка результата

Для проверки использовались выборки из таблиц и представлений под разными ролями. Например, при подключении как обычный пользователь в таблице `users` видны реальные `full_name`, `email` и `phone`. При подключении под ролью `analyst_masked` те же поля отображаются в замаскированном виде. Аналогично, материализованные представления возвращают уже обезличенные и обобщённые данные, пригодные для аналитики.

## Вывод

В ходе лабораторной работы был развёрнут `PostgreSQL Anonymizer`, настроено динамическое маскирование чувствительных полей и подготовлены два анонимизированных материализованных представления. Практически показано, что маскирование удобно использовать для ограничения доступа к исходным данным на уровне ролей, а анонимизация — для публикации и анализа агрегированной информации без раскрытия персональных и коммерчески чувствительных сведений.